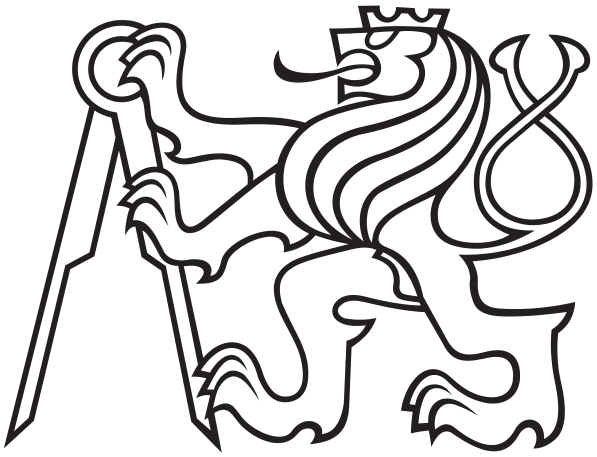
Czech Technical University in Prague
Faculty of Electrical Engineering
Department of Computer Science



Discrete Tomography

Bachelor Project

ADAM ŠULC

Study Program: Open Informatics
Project Supervisor: Mgr. Karel Chvalovský, Ph.D.

Prague, 2026

Contents

Introduction

Discrete tomography is a branch of computational imaging that focuses on reconstructing discrete images from a limited set of projections. Unlike classical tomography, where continuous images are reconstructed from dense projection data, discrete tomography deals with images composed of a finite number of intensity levels, often binary or small integer values. This restriction arises naturally in applications such as medical imaging, materials science, and industrial inspection, where objects are composed of a small set of distinguishable components.

Efficient representation and encoding of images are crucial topics in discrete tomography, as the reconstruction process depends heavily on how projection data are stored and processed. Chapter 1 introduces several encoding strategies, including standard rows, columns, and diagonal projections, but also slope-diagonal and frame-based ones. Once the image is encoded, the reconstruction problem can be translated into a set of logical constraints, which can then be solved using a SAT solver.

Discrete tomography raises fundamental theoretical questions about the uniqueness and number of possible solutions for a given set of projections. Chapter 2 addresses estimating upper bounds on the number of solutions and demonstrates how the solution space scales with the image size. Overall, theoretical analysis provides a comprehensive perspective on discrete tomography as a computational discipline.

Chapter 1

SAT Encodings of Cardinality Constraints

Cardinality constraints are a fundamental class of constraints in Boolean satisfiability problems. They restrict the number of variables that may be assigned the value true within a given set, typically enforcing conditions of the form "at most k", "at least k", or "exactly k" true variables. Such constraints naturally arise in a wide range of applications, including combinatorial reconstruction problems such as discrete tomography.

Since SAT solvers operate on formulas in conjunctive normal form (CNF), cardinality constraints must be translated into CNF using dedicated encoding techniques. A naïve translation by enumerating all forbidden combinations is generally infeasible, as it leads to an exponential grow in the number of clauses. Consequently, a variety of specialized encodings (such as the sequential counter encoding, the totalizer encoding, and cardinality networks) have been proposed to represent cardinality constraints compactly and efficiently. These encodings differ in their structural properties, the number of auxiliary variables and clauses they introduce.

In this chapter, we present an overview of these encoding techniques and discuss their theoretical characteristics. Based on Donald Knuth's book [1], particular attention is paid to their suitability for encoding constraints arising in discrete tomography problems, where large numbers of cardinality constraints are generated over overlapping variable sets.

1.1 Sequential Counter Encoding

The Sequential Counter Encoding, introduced by Carsten Sinz around 2005, is one of the most important and widely used in SAT. Let $x_1, \ldots, x_n$ be Boolean variables and let $r \in \mathbb{N}$. Sinz's method introduces $(n-r)r$ auxiliary variables

$$s_j^k \quad \text{for } 1 \le j \le n-r,\ 1 \le k \le r.$$

These variables have following interpretation: $s_j^k = 1$ means among $x_1, \ldots, x_{j+k}$ there are at least k true variables. The cardinality constraint

$$x_1 + \cdots + x_n \le r$$

is encoded by adding the following clauses:

$$(\neg s_j^k \vee s_{j+1}^k), \quad 1 \le j \le n-r-1,\ 1 \le k \le r, \tag{1.1}$$

$$(\neg x_{j+k} \vee \neg s_j^{k-1} \vee s_j^k), \quad 1 \le j \le n-r,\ 1 \le k \le r. \tag{1.2}$$

Here, s_j^0 is assumed to be true and s_j^{r+1} is omitted.

If F is any satisfiability problem and if we add the $(n-r-1)r+(n-r)(r+1)$ clauses of the forms 1.1 and 1.2, then the new set of clauses is satisfiable if and only if F is satisfiable with $x_1 + \cdots + x_n \le r$.

1.2 Totalizer Encoding

Another way to achieve the same goal, which has been proposed by O. Bailleux and Y. Boufkhad around 2003, is called Totalizer encoding. Consider a complete binary tree that has $n-1$ internal nodes numbered 1 through $n-1$, and n leaves numbered n through $2n-1$; the children of node k, for $1 \le k < n$, are nodes $2k$ and $2k+1$. We form new variables b_i^k for $1 \le k < n$ and $1 \le j \le t_k$, where t_k is the minimum of r and the number of leaves below node k. Then the following clauses do the job:

$$(\bar{b}_i^{2k} \vee \bar{b}_{i+1}^{2k+1} \vee b_{i+j}^k), \quad 0 \le i \le t_{2k},\ 0 \le j \le t_{2k+1},\ 1 \le i+j \le t_{k+1},\ 1 < k < n; \tag{1.3}$$

$$(\bar{b}_i^2 \vee \bar{b}_j^3), \quad 0 \le i \le t_2,\ 0 \le j \le t_3,\ i+j = r+1. \tag{1.4}$$

In these formulas we let $t_k = 1$ and $b_1^k = x_{k-n+1}$ for $n \le k < 2n$; all literals $\bar{b}_0^k$ and b_{r+1}^k are to be omitted. Compared to simpler encodings, Totalizer encoding offers a good trade-off between the number of auxiliary variables and clauses, while also offering strong propagation properties in SAT solvers. In particular, it supports efficient unit propagation for upper-bound constraints, making it suitable for practical SAT-based solving.

1.3 Cardinality Networks

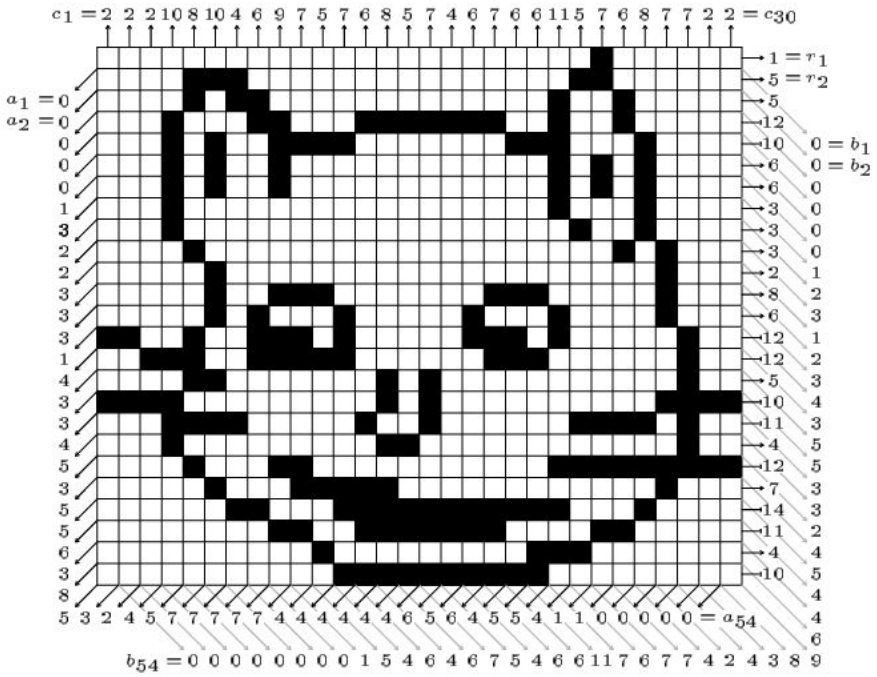
Another widely used approach for encoding cardinality constraints is based on *cardinality networks*. These encodings rely on sorting networks, such as bitonic sorting networks, to sort the input Boolean variables according to their truth values. The sorted outputs then represent the number of variables set to true, and a constraint of the form $\sum_{i=1}^{n} x_i \leq r$ can be enforced by fixing the $(r+1)$-st output of the network to false.

Cardinality network encodings are known for their strong propagation properties, which often leads to better solver performance compared to simpler encodings. However, this strength comes at the cost of a larger number of auxiliary variables and clauses, typically $\Theta(n \log^2 n)$, which can make the encoding less space-efficient than alternatives.

Chapter 2

Image Description & Runtime Analysis

In the following paragraphs we are going to focus on the study of binary images for which partial information is given. We will see how the amount of information given influences the running time and the number of solutions. As a reference example, we are going to use a "Cheshire cat" which is a bitmap of $m = 25$ rows and $n = 30$ columns.



Cheshire cat — an array of black and white pixels together with its row sums r_i, column sums c_j, and diagonal sums a_d, b_d.

2.1 Standard Image Description

Let $X = (x_{i,j}) \in \{0,1\}^{m \times n}$ be an $m \times n$ bitmap. The standard encoding consists of row, column, and diagonal projections. We are given the row sums $r_i = \sum_{j=1}^{n} x_{i,j}$ for $1 \leq i \leq m$ and the column sums $c_j = \sum_{i=1}^{m} x_{i,j}$ for $1 \leq j \leq n$, as well as both sets of sums in the 45° diagonal direction, namely $a_d = \sum_{i+j=d+1} x_{i,j}$ and $b_d = \sum_{i-j=d-n} x_{i,j}$for $0 < d < m + n$.

These vectors containing the sums can be represented by a sequence of clauses to be satisfied using one of the encodings as described earlier. The following tables compare three most popular encodings using a SAT solver Cadical195. In this case, sequential encoding is by far the best option.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	14833	28195	0.1289	0.0721
totalizer	42497	62676	0.2704	0.1519
cardnetw	36704	53987	0.1884	0.1157

Image described using r_i, c_j.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	10125	18888	0.0742	0.0379
totalizer	42655	63039	0.1992	0.3652
cardnetw	33281	48982	0.1998	0.1783

Image described using a_d, b_d.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	24208	47083	0.3207	9.3721
totalizer	84402	125715	0.4066	60.5126
cardnetw	69235	102969	0.5509	51.9776

Image described using r_i, c_j, a_d, b_d.

2.2 Slope-Diagonal Description

First, we have to carefully define slope diagonals. Let $k \in \mathbb{N}$ be a fixed slope parameter. For $d = 1, \ldots, m + \lceil \frac{n}{k} \rceil - 1$, we define the *A-slope diagonal sums* by

$$a_d^{(k)} = \sum_{\substack{1 \le i \le m \\ 1 \le j \le n \\ i + \lfloor \frac{j-1}{k} \rfloor = d}} x_{i,j}.$$

Similarly, we define the *B-slope diagonal sums* by

$$b_d^{(k)} = \sum_{\substack{1 \le i \le m \\ 1 \le j \le n \\ (m-i+1) + \lfloor \frac{j-1}{k} \rfloor = d}} x_{i,j}.$$

Note that for $k = 1$ we obtain regular diagonal sum vectors.
Suppose that we are given $a_d^{(k)}$ and $b_d^{(k)}$ for a fixed k. In the following tables, we can see how $a_d^{(k)}$ and $b_d^{(k)}$ together with row and column vectors influence the running time for various slopes k.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	13009	24621	0.0841	0.0442
totalizer	39648	58491	0.1724	0.2329
cardnetw	33405	49130	0.1664	0.1838

Image described using $a_d^{(2)}, b_d^{(2)}$.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	15235	29027	0.0922	0.0463
totalizer	38915	57334	0.1886	0.1048
cardnetw	34038	50021	0.2022	0.0942

Image described using $a_d^{(5)}, b_d^{(5)}$.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	15332	29202	0.1085	0.0542
totalizer	39304	57904	0.2020	0.1198
cardnetw	34116	50124	0.1958	0.1112

Image described using $a_d^{(10)}, b_d^{(10)}$.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	27092	52816	0.2423	13.1813
totalizer	81395	121167	0.3809	16.3476
cardnetw	69359	103117	0.4602	18.6142

Image described using $r_i, c_j, a_d^{(2)}, b_d^{(2)}$.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	29318	57222	0.2010	4.5283
totalizer	80662	120010	0.4405	19.5546
cardnetw	69992	104008	0.4139	26.8208

Image described using $r_i, c_j, a_d^{(5)}, b_d^{(5)}$.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	29415	57387	0.2094	3.4116
totalizer	81051	120580	0.4849	47.9697
cardnetw	70070	104111	0.4053	21.2280

Image described using $r_i, c_j, a_d^{(10)}, b_d^{(10)}$.

2.3 Frame-Based Description

For

$$s = 1, \ldots, \left\lceil \frac{\min(m,n)}{2} \right\rceil,$$

we define the *s-th frame sum* by

$$f_s = \sum_{\substack{s \le i \le m-s+1 \\ s \le j \le n-s+1 \\ (i=s) \vee (i=m-s+1) \vee (j=s) \vee (j=n-s+1)}} x_{i,j}.$$

The results below demonstrate what happens if we create new constraints from given sums f_s. As we have already seen in the previous section, sequential counter encoding renders best results.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	19221	36947	0.1434	0.0701
totalizer	46333	68388	0.2546	0.1277
cardnetw	35648	52361	0.1880	0.1021

Image described using f_s.

Encoding	# variables	# clauses	Build time (s)	Solve time (s)
seqcounter	33304	65142	0.2586	2.5813
totalizer	88080	131064	0.4846	35.0899
cardnetw	71692	106348	0.4174	8.8998

Image described using r_i, c_j, f_s.

2.4 Important Remarks

All of the reported runtime measurements were obtained through local testing on a personal computer. To draw reliable and statistically meaningful conclusions, it would be necessary to run these experiments on a server or, for example, on an RCI cluster.

Furthermore, it is possible to combine those descriptions and use more of them to encode an image. That would lead to decrease in number of solutions but also to a big increase in the running time. Unfortunately, I was not able to get any reasonable results on my computer.

Chapter 3

Theoretical Analysis of the Number of Solutions

In this chapter, we are going to try to estimate a number of solutions for given set of projections of an image. Because it is computationally demanding to determine the exact number of solutions for larger images, we will provide some theoretical arguments and bound the number of solutions from above, and also show that solutions grow with the size of the image.

3.1 Estimating the Upper Bound on Solutions

Our goal is to determine a generous upper bound on the possible number of different sets of input data $\{r_i, c_j, a_d, b_d\}$ that might be given to an $m \times n$ digital tomography problem, by assuming that each of those sums independently has any of its possible values. We will also show how does this bound compare to 2^{mn}, which is the total number of bitmaps of size $m \times n$. We will demonstrate our result on a bitmap with $m = 25$, $n = 30$.

We start with some basic observations. Since the length of the vectors is bounded, we obtain the following inequalities:

$$0 \leq r_i \leq n$$

$$0 \leq c_j \leq m$$

$$0 \leq a_d, b_d \leq min\{d, m, n, m + n - d\}$$

Without loss of generality, suppose that $m \leq n$. Each of the m rows consists of n pixels, so the sum of each row can take values in $\{0, 1, \ldots, n$. Thus, there are $(n + 1)$ possible values for each row sum, and consequently $(n + 1)^m$ possible distinct row sum vectors in total. Using the same reasoning,

we obtain that there are $(m+1)^n$ possible distinct column sum vectors in total.

Next, note that there are exactly $(m+n-1)$ diagonals and also anti-diagonals. Thus we only to compute the number of possibilities for diagonals and then simply square it. A diagonal with index d contains exactly $min\{d, m, n, m+n-d\}$ pixels. Thus, the total number of possible distinct diagonal sum vectors is

$$\prod_{d=1}^{m+n-1} (1 + min\{d, m, n, m+n-d\})$$

We can further simplify this expression knowing that the lengths are $1, 2, \ldots, m, m, \ldots, m, m-1, \ldots, 2, 1$. Thus, the product becomes

$$\prod_{k=1}^{m} (k+1) \cdot (m+1)^{n-m} \cdot \prod_{k=1}^{m-1} (k+1)$$

Using factorials, we can express the number of possibilities as

$$(m+1)! \cdot (m+1)^{n-m} \cdot m!$$

Putting it all together, we have shown that the total number B of possible combinations of row, column, and diagonal sums is

$$(n+1)^m (m+1)^n ((m+1)! (m+1)^{n-m} m!)^2$$

For $m = 25$ and $n = 30$, one can compute that $B \approx 3 \times 10^{197}$. If we roughly assume that bitmaps are "evenly distributed" among tomography vectors, then an application of the pigeonhole principle shows that the average number of bitmaps per tomography vector is $\frac{2^{mn}}{B}$. This estimate is not exact but is intended to be informative. Since $\frac{2^{750}}{B} \approx 2 \cdot 10^{28}$, we can conclude that a "random" 25×30 digital tomography problem typically has more than 10^{28} solutions.

The following goal is to show the what happens if we use slope diagonals instead the regular ones. Let $k \in \mathbb{N}$ be a fixed slope parameter. By definition, an image contains exactly $(m + \lceil \frac{n}{k} \rceil - 1)$ slope diagonals (and also slope anti-diagonals). Assume that $m \leq n$ and define $I = min\{m-1, \lfloor \frac{n-1}{k} \rfloor\}$. This number represents how many slope diagonals increase in length before reaching the maximum length $L = I \cdot k + min\{k, n - I \cdot k\}$. We can divide the slope diagonals into 3 categories: increase phase, plateau phase and decrease phase. First, there are exactly I slope diagonals in the increase phase and their lengths are $k, 2k, \ldots, I \cdot k$. Next comes the plateau phase which contains exactly P slope diagonals with length L, where

$$P = \begin{cases} m - I, & \text{if } I < m-1, \\ \lceil \frac{n}{k} \rceil - I, & \text{otherwise.} \end{cases}$$

Lastly, exactly D slope diagonals belong to the decrease phase and their lengths are $L-k, L-2k, \cdots$, where

$$D = \begin{cases} \lceil \frac{n}{k} \rceil - 1, & \text{if } I < m-1, \\ \lceil \frac{n}{k} \rceil - P, & \text{otherwise.} \end{cases}$$

Now we can express the number of possible distinct slope-diagonal sum vector as

$$\prod_{l=1}^{I} kl \cdot L^P \cdot \prod_{l=1}^{D} (L - kl)$$

For $m = 25$, $n = 30$ and $k = 2$, we can easily compute that $I = 14, L = 30, P = 11$ and $D = 14$. Using the expressions for number of row and column sum vectors computed above, we finally obtain $B \propto 10^{126}$. Since $\frac{2^{750}}{B} \propto 10^{99}$, we can conclude that a "random" 25×30 digital tomography problem encoded using row, column and slope diagonal sums with $k = 2$ typically has more than 10^{99} solutions.

In the last part, we substitute the diagonals with frames. As before, assume that $m \leq n$. Observe that a s-th frame f_s contains exactly $(2m + 2n - 8s + 4$ pixels, for $s = 1, \ldots, \lceil \frac{m}{2} \rceil$. Thus, the we can express the number of frame sum vectors as

$$\prod_{s=1}^{\lceil \frac{m}{2} \rceil} (2m + 2n - 8s + 4)$$

Using the number of possibilities for row and column sums computed previously, we obtain $B \propto 10^{103}$. Since $\frac{2^{750}}{B} \propto 10^{122}$, we can conclude that a "random" 25×30 digital tomography problem encoded using row, column and frame sums typically has more than 10^{122} solutions.

3.2 Relation Between Number of Solutions and Image Size

In this section, we restrict to row, column and diagonal projections. We will show that the expected number of solutions of an $m \times n$ tomography instance increases with m and n. Note that this is an average-case statement, we cannot guarantee that for every instance larger images have more solutions.

To prove this, we will use two estimates computed in the precedent section. Since each pixel take value 0 or 1, the total number of binary images is 2^{mn}. The number T of distinct tomography vectors $\leq B(m,n)$, because $B(m,n)$ counts all syntactically admissible vectors, including unrealizable ones. By applying a pigeonhole principle, we might deduce that the average number of solutions $= \frac{2^{mn}}{T} \geq \frac{2^{mn}}{B(m,n)}$. Showing $\frac{2^{mn}}{B(m,n)} \to \infty$ as $m, n \to \infty$ is equivalent to the desired statement.

To simplify it a little, we will use logarithms. Since the logarithm is monotone and unbounded, it is sufficient to show that $\log(\frac{2^{mn}}{B(m,n)}) = \log(2^{mn}) - \log(B(m,n)) \to \infty$ as $m, n \to \infty$. It is not too difficult to show $\log(2^{mn}) = \Theta(mn)$. However, the second term is a bit more complicated to handle.

$$\log B(m,n) = m\log(n+1) + n\log(m+1) + 2\log((m+1)!) + 2(n-m)\log(m+1) + 2\log(m!)$$

In the next step, we apply a Stirling approximation formula: $\log k! = k\log k - k + \mathcal{O}(\log k)$.

$$\begin{aligned} m\log(n+1) &= \Theta(m\log n) \\ n\log(m+1) &= \Theta(n\log m) \\ 2\log((m+1)!) &= \Theta(m\log m) \\ 2(n-m)\log(m+1) &= \Theta(n\log m) \\ 2\log(m!) &= \Theta(m\log m) \end{aligned}$$

Thus, we can write $\log B(m,n) = \Theta(m\log n + n\log m + m\log m)$. Since $\log m \leq \log(m+n)$ and $\log n \leq \log(m+n)$, the following inequalities holds

$$\begin{aligned} m\log n &\leq m\log(m+n) \\ n\log m &\leq n\log(m+n) \\ m\log m &\leq m\log(m+n) \end{aligned}$$

Summing them up yields $\log B(m,n) = \mathcal{O}((m+n)\log(m+n))$. Since $mn \gg (m+n)\log(m+n)$, $\log(\frac{2^{mn}}{B(m,n)}) = \Theta(mn) - \mathcal{O}((m+n)\log(m+n)) \to \infty$ as $m, n \to \infty$. That implies the desired fact $\frac{2^{mn}}{B(m,n)} \to \infty$ as $m, n \to \infty$.

To conclude, the number of images grows much faster than the number of distinct tomography constraints. Therefore, larger images have on average more solutions than smaller ones.

Bibliography

[1] Donald E. Knuth. *The Art of Computer Programming.* Addison-Wesley, 3rd edition, 1997.